

Week 5 Assignment: A Market with Memory

Overview

In this assignment you will do four things:

1. **Give the market structure** by extending your trading environment with an AR(1) return process — for the first time, there is something for the agent to learn.
2. **Verify the signal exists** statistically, before training anything on it.
3. **Train PPO on the momentum market** and show that it finally beats the random and buy-and-hold baselines, using the Week 4 evaluation harness unchanged.
4. **Explore the edge-vs-cost trade-off** — how signal strength and transaction cost together decide whether trading is worth it.

Setup

No new packages this week. Confirm your Week 4 files still run:

```
python week4_evaluate.py # should train and produce plots/cumulative_pnl.png
```

You will import `average_cumulative_pnl` and `plot_pnl` from `week4_evaluate.py`, so keep it in the same folder.

Project Structure

By the end of Week 5, your folder should look like:

```
project_root/
  week4_trading_env_v2.py      (from Week 4)
  week4_evaluate.py           (from Week 4)
  week5_trading_env_v3.py     (Task 1)
  week5_check_signal.py       (Task 2)
  week5_train_and_evaluate.py (Task 3)
  week5_sweep.py              (Task 4)
  week5_notes.md              (Task 5)
  plots/
    cumulative_pnl.png        (from Week 4)
    cumulative_pnl_momentum.png (Task 3)
```

Task 1: An AR(1) Trading Environment (Core)

The Week 4 environment drew each price return independently — a pure random walk, which is unbeatable by design. This week the *only* change is one line in the price dynamics: each return now partly remembers the previous one,

$$r_t = \rho r_{t-1} + \varepsilon_t, \quad \varepsilon_t \sim \mathcal{N}(0, \sigma^2).$$

Create `week5_trading_env_v3.py` using this skeleton:

```

# week5_trading_env_v3.py

import gymnasium as gym
from gymnasium import spaces
import numpy as np
from collections import deque

class TradingEnvV3(gym.Env):
    """
    Trading environment with an AR(1) return process.

    Price dynamics:
         $r_t = \rho * r_{t-1} + \epsilon_t$ ,  $\epsilon_t \sim N(0, \text{daily\_vol})$ 

         $\rho > 0$  -> momentum (returns tend to repeat their sign)
         $\rho = 0$  -> random walk (identical to Week 4)
         $\rho < 0$  -> mean reversion (returns tend to flip their sign)

    Observation (history_len + 1 dimensional):
        [r_{t-history_len+1}, ..., r_t, current_position]

    Actions (discrete):
        0 = Go Long, 1 = Hold, 2 = Go Short

    Reward:
        position * price_return * 100
        - transaction_cost * |new_position - old_position|
    """

    def __init__(
        self,
        episode_length=100,
        initial_price=100.0,
        daily_vol=0.01,
        history_len=5,
        transaction_cost=0.1,
        rho=0.5,
    ):
        super().__init__()
        assert -1.0 < rho < 1.0, "rho must be in (-1, 1) or returns explode"
        self.episode_length = episode_length
        self.initial_price = initial_price
        self.daily_vol = daily_vol
        self.history_len = history_len
        self.transaction_cost = transaction_cost
        self.rho = rho

        obs_dim = history_len + 1
        self.observation_space = spaces.Box(
            low=np.full(obs_dim, -np.inf, dtype=np.float32),
            high=np.full(obs_dim, np.inf, dtype=np.float32),
        )
        self.action_space = spaces.Discrete(3)

        self.price = None
        self.position = None
        self.step_count = None

```

```

        self._price_history = None
        self._last_return = None

# ----- #
def reset(self, seed=None, options=None):
    super().reset(seed=seed)
    self.price = self.initial_price
    self.position = 0.0
    self.step_count = 0
    self._last_return = 0.0
    self._price_history = deque(
        [0.0] * self.history_len, maxlen=self.history_len
    )
    return self._get_obs(), {}

# ----- #
def step(self, action):
    # AR(1) price dynamics -- the ONLY change from Week 4
    noise = float(self.np_random.normal(0.0, self.daily_vol))
    price_return = self.rho * self._last_return + noise
    # TODO: add a comment explaining what this line does when
    # rho > 0, rho = 0, and rho < 0
    self._last_return = price_return

    self.price *= (1.0 + price_return)
    self._price_history.append(price_return)

    action_to_position = {0: 1.0, 1: self.position, 2: -1.0}
    new_position = action_to_position[int(action)]

    pnl = self.position * price_return * 100.0
    cost = self.transaction_cost * abs(new_position - self.position)
    reward = pnl - cost

    self.position = new_position
    self.step_count += 1
    terminated = self.step_count >= self.episode_length
    truncated = False
    return self._get_obs(), reward, terminated, truncated, {}

# ----- #
def _get_obs(self):
    history = np.array(list(self._price_history), dtype=np.float32)
    return np.append(history, self.position).astype(np.float32)

# ----- #
def render(self):
    print(
        f"Step {self.step_count:3d} | "
        f"Price {self.price:8.2f} | "
        f"Position {self.position:+.0f}"
    )

```

What You Must Do

1. Read through the skeleton and fill in the one TODO comment in `step()` in your own words.
2. Run the usual sanity check and fix any warnings:

```
from gymnasium.utils.env_checker import check_env
from week5_trading_env_v3 import TradingEnvV3
check_env(TradingEnvV3(), warn=True)
```

3. Confirm that `TradingEnvV3(rho=0.0)` behaves exactly like Week 4's environment: run 3 random-agent episodes and check the rewards look similar in scale.

Task 2: Verify the Signal Exists (Core)

Before training an agent, verify that the structure you *think* you added is really there. This habit — check the data before blaming the algorithm — will matter even more next week with real data.

Create `week5_check_signal.py`:

```
# week5_check_signal.py

import numpy as np
from week5_trading_env_v3 import TradingEnvV3

def collect_returns(env, n_steps=10_000):
    """Step through the env with Hold actions and record each return."""
    returns = []
    obs, _ = env.reset(seed=0)
    for _ in range(n_steps):
        obs, reward, terminated, truncated, _ = env.step(1) # Hold
        returns.append(obs[-2]) # newest return in the history window
        if terminated or truncated:
            obs, _ = env.reset()
    return np.array(returns)

def lag1_autocorrelation(returns):
    """Correlation between each return and the one right after it."""
    return np.corrcoef(returns[:-1], returns[1:])[0, 1]

def random_agent_mean_reward(env, n_episodes=20):
    totals = []
    for _ in range(n_episodes):
        obs, _ = env.reset()
        total, done = 0.0, False
        while not done:
            obs, reward, terminated, truncated, _ = env.step(
                env.action_space.sample()
            )
            total += reward
            done = terminated or truncated
        totals.append(total)
    return float(np.mean(totals))

if __name__ == "__main__":
    for rho in [0.0, 0.5, -0.5]:
        env = TradingEnvV3(rho=rho, transaction_cost=0.0)
        r = collect_returns(env)
        ac = lag1_autocorrelation(r)
```

```

mean_rand = random_agent_mean_reward(TradingEnvV3(rho=rho))
print(f"rho = {rho:+.2f} | measured autocorr = {ac:+.3f} | "
      f"random agent mean reward = {mean_rand:+8.2f}")

```

What You Must Do

1. Read through the skeleton and add comments in your own words explaining:
 - Why the returns are collected with Hold actions (would the numbers change with a different action sequence?).
 - What `np.corrcoef(returns[:-1], returns[1:])` is comparing.
2. Run the script and record the three measured autocorrelations. They should land close to the `rho` you set (within a few hundredths).
3. Note the random agent's mean reward for each `rho`. It should come out slightly *negative* (around -8 with the default settings) — and, the key point, it should be roughly the *same* whether the signal exists or not. Think about both facts — you will write them up in Task 5.

Task 3: Train PPO on the Momentum Market (Core)

Now train PPO on the momentum market and evaluate it exactly the way you evaluated the random-walk agent in Week 4. Create `week5_train_and_evaluate.py`:

```

# week5_train_and_evaluate.py

import numpy as np
from stable_baselines3 import PPO
from week5_trading_env_v3 import TradingEnvV3
from week4_evaluate import average_cumulative_pnl, plot_pnl

if __name__ == "__main__":
    env = TradingEnvV3(rho=0.5, transaction_cost=0.1)

    print("Training PPO on the momentum market (rho = 0.5) ...")
    model = PPO(
        "MlpPolicy", env, verbose=0,
        learning_rate=3e-4, n_steps=512,
        batch_size=64, gamma=0.99,
    )
    model.learn(total_timesteps=100_000)
    print("Training complete.")

    env_eval = TradingEnvV3(rho=0.5, transaction_cost=0.1)
    pnl_trained = average_cumulative_pnl(
        env_eval, n_episodes=50, model=model, strategy="trained")
    pnl_random = average_cumulative_pnl(
        env_eval, n_episodes=50, model=None, strategy="random")
    pnl_bnh = average_cumulative_pnl(
        env_eval, n_episodes=50, model=None, strategy="buy_and_hold")

    print("Final cumulative P&L (mean over 50 episodes):")
    print(f"  PPO trained:  {pnl_trained[-1]:+8.2f}")

```

```

print(f" Random agent: {pnl_random[-1]:+8.2f}")
print(f" Buy and hold: {pnl_bnh[-1]:+8.2f}")

plot_pnl(pnl_trained, pnl_random, pnl_bnh,
         filename="plots/cumulative_pnl_momentum.png")

```

What You Must Do

1. Run the script and save `plots/cumulative_pnl_momentum.png`. This time the PPO curve should pull clearly above both baselines. If it does not, see Common Issues at the end.
2. Put this plot side by side with Week 4's `cumulative_pnl.png`. The *only* difference between the two experiments is ρ . Write down what changed — you will need it for Task 5.
3. Inspect what the agent actually learned. Run one episode step by step, and after each step print the newest return and the position the agent chose:

```

obs, _ = env_eval.reset()
done = False
while not done:
    action, _ = model.predict(obs, deterministic=True)
    obs, reward, terminated, truncated, _ = env_eval.step(action)
    print(f"last return {obs[-2]:+.4f} -> position {obs[-1]:+.0f}")
    done = terminated or truncated

```

In Week 1 you beat a random CartPole agent with a hand-coded rule. For this market you can guess the sensible hand-coded rule (“hold the position that matches the sign of the last return”). Does the agent’s behaviour look like that rule? Note any differences — for example, what it does when the last return is very small.

Task 4: The Edge-vs-Cost Trade-off (Core)

A signal is only worth trading if it is stronger than the cost of trading it. Create `week5_sweep.py`:

```

# week5_sweep.py

import numpy as np
from stable_baselines3 import PPO
from week5_trading_env_v3 import TradingEnvV3
from week4_evaluate import average_cumulative_pnl

def train_and_score(rho, transaction_cost, total_timesteps=50_000):
    """Train PPO, return the final mean cumulative P&L over 50 episodes."""
    env = TradingEnvV3(rho=rho, transaction_cost=transaction_cost)
    model = PPO("MlpPolicy", env, verbose=0,
                learning_rate=3e-4, n_steps=512,
                batch_size=64, gamma=0.99)
    model.learn(total_timesteps=total_timesteps)
    env_eval = TradingEnvV3(rho=rho, transaction_cost=transaction_cost)
    pnl = average_cumulative_pnl(env_eval, n_episodes=50,
                                model=model, strategy="trained")

    return pnl[-1]

```

```

if __name__ == "__main__":
    print("--- Sweep 1: signal strength (transaction_cost = 0.1) ---")
    for rho in [0.0, 0.25, 0.5]:
        score = train_and_score(rho, transaction_cost=0.1)
        print(f"rho = {rho:.2f} | final mean cumulative P&L = {score:+8.2f}")

    print("--- Sweep 2: transaction cost (rho = 0.25) ---")
    for cost in [0.0, 0.1, 0.3]:
        score = train_and_score(0.25, transaction_cost=cost)
        print(f"cost = {cost:.2f} | final mean cumulative P&L = {score:+8.2f}")

```

What You Must Do

1. Run the script (six short training runs at 50 000 timesteps each — expect several minutes on a laptop CPU; this is a comparison, not a benchmark, so the shorter runs are fine).
2. Record the six numbers in a small table in your notes. You are looking for two patterns: P&L should *grow* with `rho` in Sweep 1, and *shrink* as `cost` rises in Sweep 2.
3. These numbers are noisy single training runs. If one of them looks out of place, re-run that configuration once and note how much the result moved. This is your first taste of why serious RL papers report averages over multiple seeds.

Task 5: Written Reflection (`week5_notes.md`)

Create `week5_notes.md` and answer each question in 2–4 sentences.

On the signal check (Task 2):

1. What were your three measured autocorrelations? Why do they not match `rho` exactly?
2. The random agent's mean reward was slightly negative and roughly the same for all three values of ρ . What causes the negative drift (hint: how often does a random agent change position, and what does each change cost)? And why does market structure alone not help an agent that ignores the state?

On training (Task 3):

3. Compare `cumulative_pnl_momentum.png` to Week 4's `cumulative_pnl.png`. What changed, and what single property of the environment explains the difference?
4. Describe the policy you observed in Task 3 step 3. How close is it to the hand-coded rule “match the sign of the last return”? Did the transaction cost visibly change its behaviour (e.g., holding through small wiggles)?
5. Buy-and-hold ends near zero on this market even though momentum exists. Why can a *fixed* position not exploit autocorrelation?

On the sweep (Task 4):

6. How did final P&L change with `rho`? Is the relationship what you expected?
7. At `rho=0.25`, roughly where did the transaction cost eat the whole edge? Explain the trade-off in one or two sentences, as if to a friend who has never heard of RL.

8. Suppose $\rho = -0.5$ (mean reversion). Without training anything, what should the optimal position rule be? (You can check your answer in the optional extension.)

Big picture:

9. Next week you will measure the lag-1 autocorrelation of *real* daily stock returns. Do you expect it to be closer to 0.5 or to 0.0? What does your guess imply about how hard next week will be?
10. Write down **3 questions** about market structure or RL-for-trading that you want to discuss with your mentor.

Optional Extension: The Mean-Reversion Market

If you are ahead of pace, train a fresh PPO agent on `TradingEnvV3(rho=-0.5, transaction_cost=0.1)` and evaluate it as in Task 3. Then do the interesting part: take the agent you trained on the *momentum* market and evaluate it (without retraining) on the mean-reversion market. It should lose money — it learned exactly the wrong rule for this world. This is your first encounter with *distribution shift*: an agent is only as good as the match between its training world and its deployment world. Add a short note on what you observe in `week5_notes.md`.

Deliverables

By the end of Week 5, you should have:

- **Code:** `week5_trading_env_v3.py`, `week5_check_signal.py`, `week5_train_and_evaluate.py`, `week5_sweep.py`.
- **Notes:** `week5_notes.md` with answers to all 10 questions and the sweep table.
- **Plot:** `plots/cumulative_pnl_momentum.png`.

During your mentor meeting, walk through:

- The momentum P&L plot next to Week 4’s random-walk plot — the before/after story of adding structure.
- Your sweep table — where the edge beats the cost and where it does not.
- Your answer to Question 9 — your prediction for real data.
- The 3 questions from Question 10 in your notes.

Common Issues

“PPO does not beat the baselines even at `rho=0.5`.”

First check Task 2’s output — is the measured autocorrelation actually near 0.5? If yes, train longer (`total_timesteps=200_000`) and confirm your observation includes the most recent return (print an observation and check). If the agent trades but loses, your `transaction_cost` may be too high relative to `daily_vol`.

“The measured autocorrelation is 0.48, not 0.50.”

That is sampling noise over 10 000 steps, plus a small effect from episode resets (each `reset()` clears `_last_return`, breaking the chain at episode boundaries). Both are expected; do not chase the third decimal place.

“My sweep numbers jump around between runs.”

Each entry is a single training run with its own random seed, so some spread is normal. The *trend* across `rho` and `cost` should still be visible. If it is not, increase `total_timesteps` or evaluate over more episodes.

“The agent flips position every single step at cost=0.0.”

That is rational: with no cost, there is no reason not to re-optimize the position every step. Watch how the behaviour changes as the cost rises — that contrast is the point of Sweep 2.